

Fake News Detection with Transformers

Overview

This project aims to tackle the growing threat of misinformation by building a robust **fake news detection system**. Leveraging **transformer-based NLP models**, particularly **RoBERTa**, we classify online articles as *Fake (0)* or *Real (1)* with high accuracy.

Dataset: 15,116 labeled news articles

Final Model: Fine-tuned RoBERTa Transformer

Metric: Macro-averaged F1-score

Objectives

- 1. **Build baseline and ensemble models** for classification.
- 2. **Develop LSTM neural network** to capture sequential text patterns.
- 3. **Fine-tune RoBERTa transformer** for contextual performance.
- 4. **Compare model performances** and select the best one for deployment.
- 5. **Deploy using Docker, FastAPI, and Streamlit.**

Models Built

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	82.27%	83.17%	96.18%	89.20%	81.85%
XGBoost	82.60%	84.36%	94.70%	89.23%	82.19%
LSTM	82.14%	85.84%	91.66%	88.65%	80.56%
RoBERTa (Best)	84.70%	86.91%	94.06%	90.35%	85.58%

Project Structure

graphql

CopyEdit

fake-news-detection/

```
├── data/          # Cleaned and preprocessed dataset
├── notebooks/     # Jupyter notebooks for EDA and modeling
├── deployment/
|   ├── backend/   # FastAPI server (main.py)
|   └── frontend/  # Streamlit interface (app.py)
├── roberta_model/ # Fine-tuned RoBERTa model + tokenizer
├── Dockerfile     # Backend container
├── Dockerfile.frontend # Frontend container
├── docker-compose.yml # Compose setup for both services
└── README.md      # Project documentation
```

Tech Stack

- **Python:** Data processing, modeling, backend & frontend scripting
- **Libraries:** scikit-learn, tensorflow, torch, transformers, spaCy, gensim, lime
- **Web App:** FastAPI, Streamlit
- **Deployment:** Docker, Docker Compose

Data Pipeline

1. Loading & Cleaning

- Removed duplicates, handled missing values
- Preprocessed text using SpaCy

2. Feature Engineering

- Extracted domains, text length, sentence count
- Tokenization, lemmatization, and Word2Vec embeddings

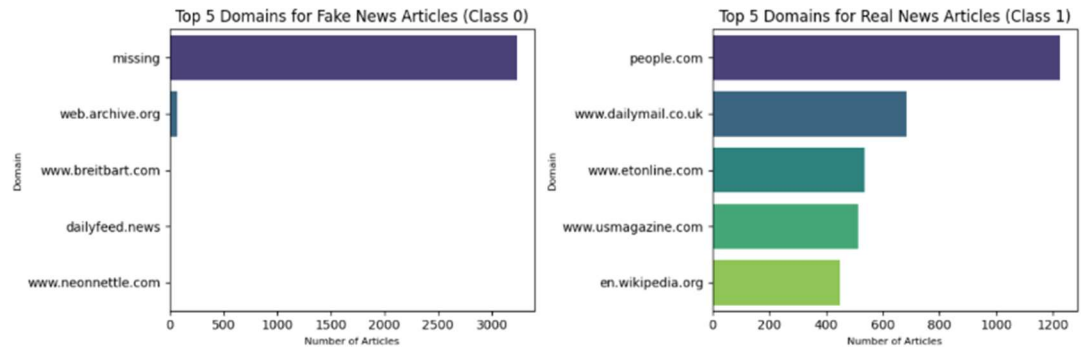
3. Exploratory Data Analysis (EDA)

-



-





○

4. Modeling Approaches

- Logistic Regression (TF-IDF + Word2Vec)
- XGBoost (Ensemble)
- LSTM Neural Network
- Fine-tuned RoBERTa

5. Evaluation Metrics

- F1, Precision, Recall, ROC-AUC
- Confusion matrices
- ROC curves for all models

Model Interpretability

- Used **LIME** (Local Interpretable Model-Agnostic Explanations)
- Visualized token-level importance for predictions from RoBERTa
- Interpretations guide model trust and transparency

Deployment

- **FastAPI backend** loads the model and returns predictions + LIME explanations.
- **Streamlit frontend** lets users paste a URL and view predictions + highlighted words.
- **Dockerized** using two containers:
 - 8000: FastAPI backend

- 8501: Streamlit frontend

Thresholding Strategy

- Due to class imbalance, custom thresholds were applied:
 - Fake Article (Label 0) → threshold = 0.30
 - Real Article (Label 1) → threshold = 0.75
- Articles with probabilities in the gray zone are flagged for **human review**.

Conclusion

- **Finetuned RoBERTa** was the top-performing model with strong generalization.
- The model is explainable and scalable—ideal for **real-world deployment** on platforms like **AWS SageMaker**, **browser extensions**, or **social media APIs**.

Future Steps

- Deploy on **AWS SageMaker** for production-ready inference
- Create **browser plugins** for live credibility scoring
- Automate flagging of ambiguous predictions for **human validation**